EE6406: Analytic and Ensemble Learning

Lecture 3
Linear Parametric Models

Overview

1. Introduction
2. Systems of Linear Equations
3. Models with Linear Parameters
4. Polynomials
5. Reduced Multivariate Polynomials
6. Kernel Models

Introduction

- Learning from data plays a key role in the fields of statistics, artificial intelligence and many areas of science and engineering.
- A typical task in such applications is to predict the future *outcome* based on a collection of historical *observations* or *measurements* .
- Under such a learning paradigm, the outcome is often called the *response* or the *output* of the system. The observations or measurements are also termed *features* or *inputs* to the system.

Introduction

- The system is often modeled by a mathematical function which relates the features or inputs to the output.
- The task of learning is to find a mathematical model which predicts the responses of novel observations or measurements.
- We shall focus on the many ways of formulating such a mathematical model, particularly a linear one with respect to its parameters, assuming that it exists and describes the system relatively well.
- It is noted here that the linearity is only with respect to the parameters where nonlinearity can still live between the inputs and the outputs.

Learning Algorithms

In general, learning algorithms consist of the following components:

- a loss function;
- an optimization criterion (also known as an objective function or a cost function) based on the loss function; and
- an optimization routine leveraging training data to find a solution to the optimization criterion.

Learning Components

$$\arg\min_{\boldsymbol{w}} J(\boldsymbol{w}) = \arg\min_{\boldsymbol{w}} \sum_{i=1}^{m} L(g(\boldsymbol{x}_i, \boldsymbol{w}), y_i) + \lambda R(\boldsymbol{w})$$

where

$J(\boldsymbol{w})$	:	Learning cost function (optimization criterion)
$L(\cdot)$	:	Loss function
$g(\boldsymbol{x}_i, \boldsymbol{w})$	:	Learning model
$\mathbf{x}_i$	:	Learning model input vector
$\boldsymbol{w}$	:	Learning parameter vector
y_i	:	Learning target
m	:	Sample size
$R(\boldsymbol{w})$	:	Regularization function
λ	:	Regularization factor

Illustration

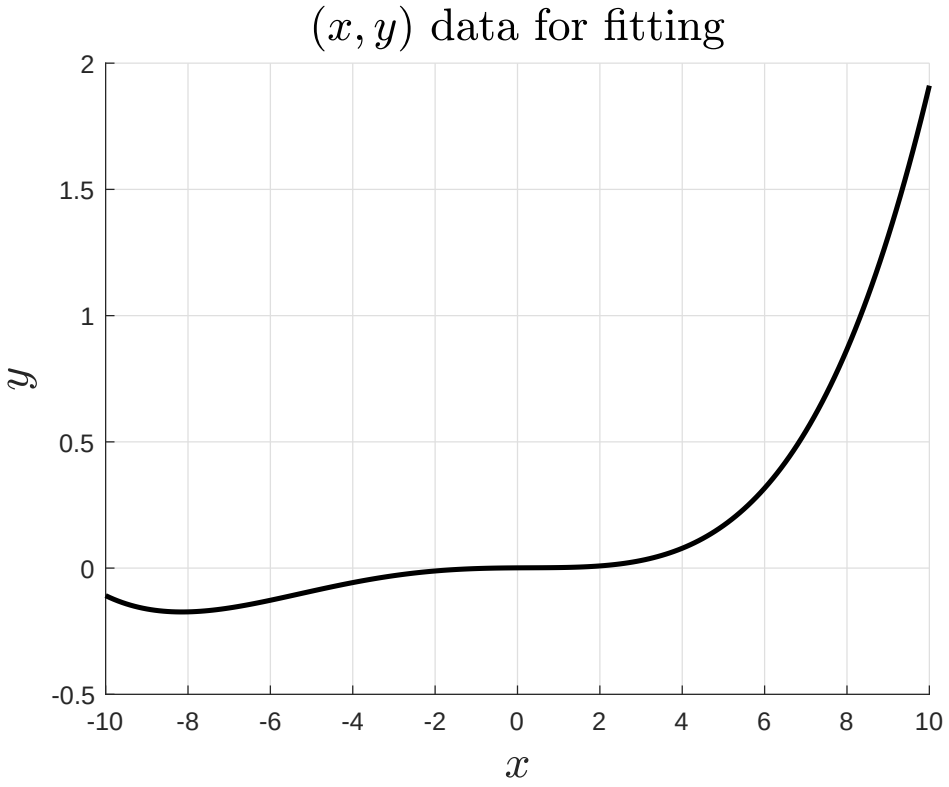

Figure 1: Data for learning.

Illustration (cont'd)

$$\min_{w} \sum_{i=1}^{m} (g_w(x_i) - y_i)^2, \quad g_w(x_i) = g(x_i, w) = w x_i$$

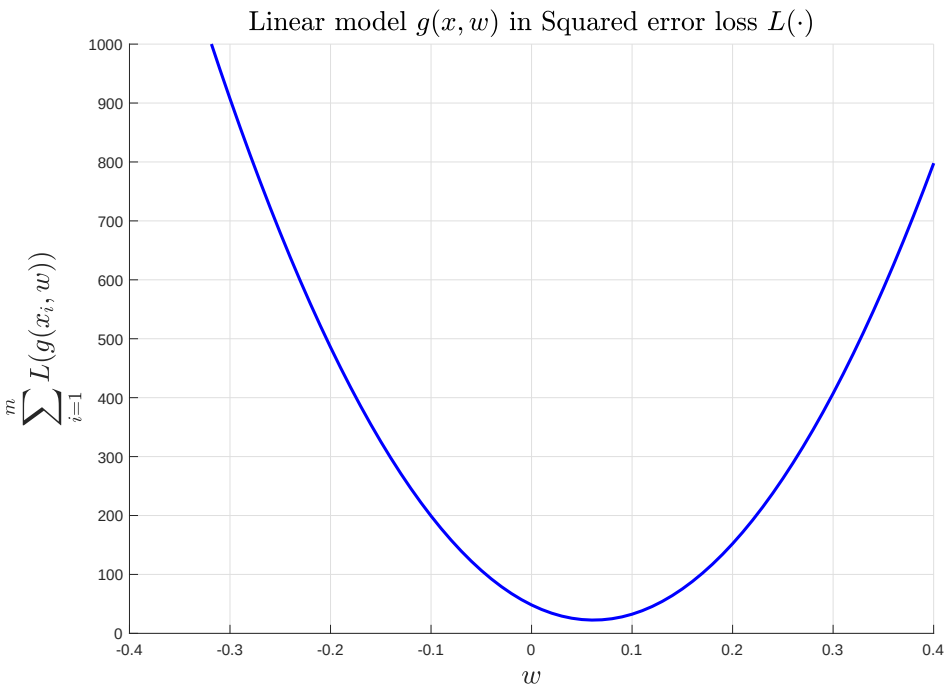

Figure 2: Learning cost function.

Illustration (cont'd)

$$\min_{w} \sum_{i=1}^{m} (g_w(x_i) - y_i)^2, \quad g_w(x_i) = \sigma(wx_i),\ \sigma(a) = \max(0, a)$$

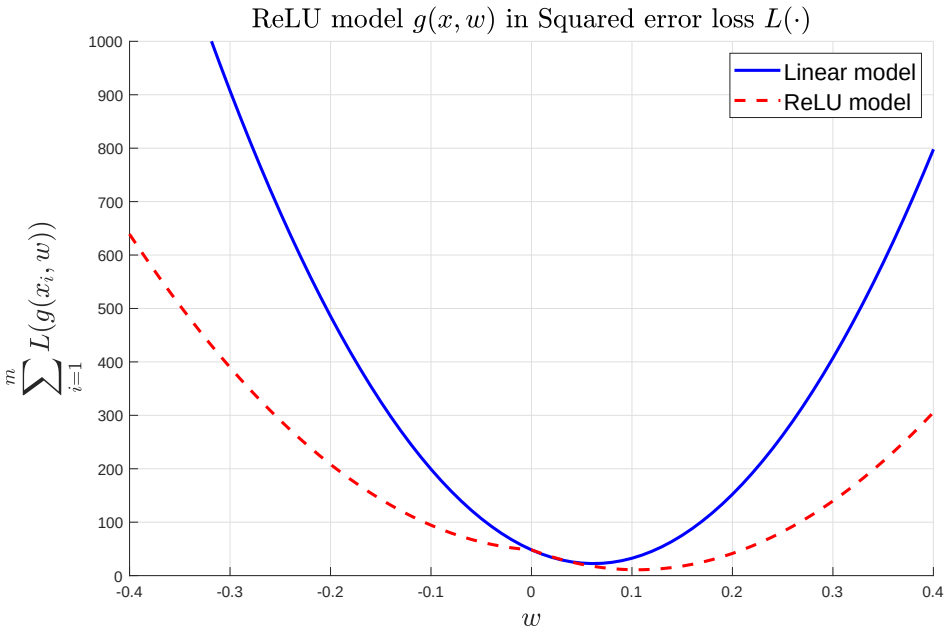

Figure 3: Learning cost function.

Notations

- Throughout the lecture, a scalar variable uses an non-boldface case letter such as x or X and a vectored variable uses a boldface lower case letter such as $\boldsymbol{x}$ or $\mathbf{x}$.
- For more specificity, $\mathbb{R}^d$ indicates that there are d number of real elements in the vector.
- Unless otherwise specified, a vector is taken to be a column vector by default.
- For a matrix, an upper case letter in boldface such as $\mathbf{X}$ is denoted.
- The dimension of a matrix is indicated by specifying the row size followed by the column size. For example, $\mathbf{X} \in \mathbb{R}^{m \times d}$ indicates a matrix $\mathbf{X}$ of real numbers with m rows and d columns.

Systems of Linear Equations

Consider a linear equation with d variables given by

$$y = x_1 w_1 + x_2 w_2 + \cdots x_d w_d \tag{3.1}$$

where $x_1, \cdots x_d$ are the observed variables, $w_1, \cdots w_d$ are fixed but unknown weighting constants, and y is the system response. When the values of $x_1, \cdots x_d$ are varied, the system responses with corresponding y values. Suppose we have a collection of m variations of these $x_1, \cdots x_d$ values with corresponding y values, then we can write them as

$$\begin{aligned} x_{1,1} w_1 + x_{1,2} w_2 + \cdots + x_{1,d} w_d &= y_1 \\ x_{2,1} w_1 + x_{2,2} w_2 + \cdots + x_{2,d} w_d &= y_2 \\ &\vdots \\ x_{m,1} w_1 + x_{m,2} w_2 + \cdots + x_{m,d} w_d &= y_m. \end{aligned} \tag{3.2}$$

Systems of Linear Equations

Let

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \cdots & x_{m,d} \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \tag{3.3}$$

then the set of equations in (3.2) can be written in matrix-vector form:

$$\mathbf{X}\boldsymbol{w} = \mathbf{y}. \tag{3.4}$$

Depending on the sizes of m and d, different types of solutions are available for this system. We shall consider them separately in the following discussions.

Square or Even-determined System

When the sample size is equal to the variable dimension i.e., $m = d$, we have a square system (also known as even-determined system) with equal number of equations and unknowns. If $\mathbf{X}$ is invertible so that $\mathbf{X}^{-1}\mathbf{X} = \mathbf{I}$ with $\mathbf{I}$ being an identity matrix, then by pre-multiplying both sides of (3.4) by $\mathbf{X}^{-1}$, we have a unique solution given by

$$\begin{aligned} \mathbf{X}^{-1}\mathbf{X}\boldsymbol{w} &= \mathbf{X}^{-1}\mathbf{y} \\ \Rightarrow \quad \hat{\boldsymbol{w}} &= \mathbf{X}^{-1}\mathbf{y}. \end{aligned} \tag{3.5}$$

Here, a hat has been placed on top of $\boldsymbol{w}$ to indicate that $\hat{\boldsymbol{w}}$ is a specific point in the space of $\boldsymbol{w}$.

Square or Even-determined System

Example 3.1

Consider the following system of two equations with two unknowns,

$$w_1 + 2w_2 = 5 \tag{3.6}$$

$$3w_1 - 4w_2 = 6, \tag{3.7}$$

which can be packed into matrix-vector form $\mathbf{X}\boldsymbol{w} = \mathbf{y}$ where

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & -4 \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

According to (3.5), and since $\mathbf{X}$ is invertible, we have

$$\hat{\boldsymbol{w}} = \mathbf{X}^{-1}\mathbf{y} = \begin{bmatrix} 1 & 2 \\ 3 & -4 \end{bmatrix}^{-1} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 3.2 \\ 0.9 \end{bmatrix}. \tag{3.8}$$

Square or Even-determined System

Example 3.1 cont'd

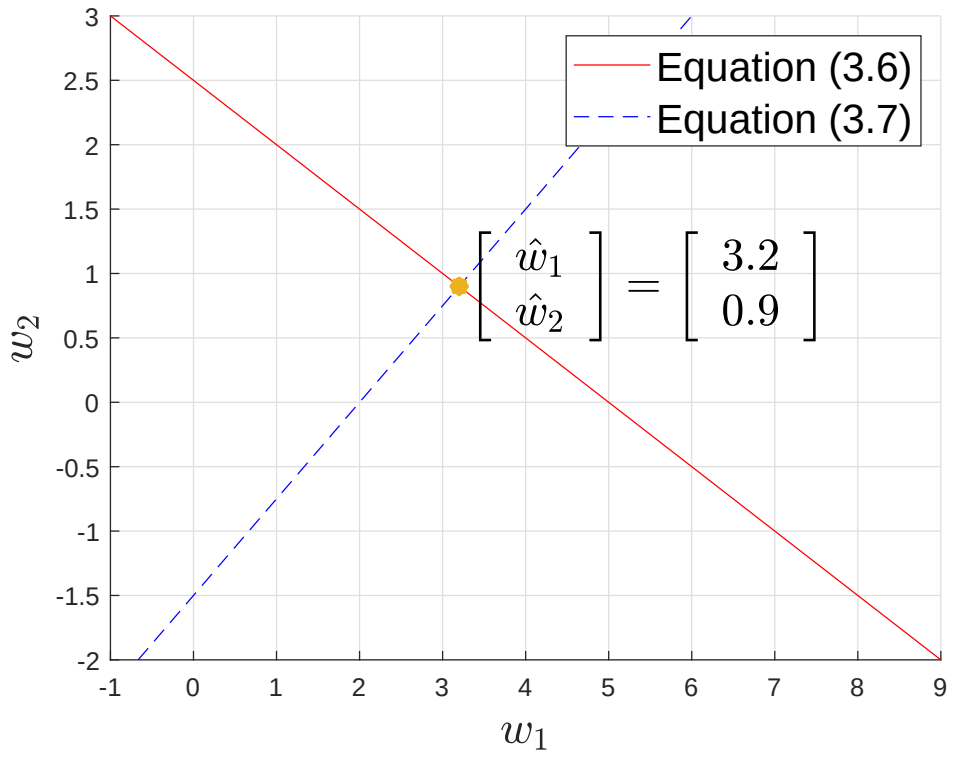

Figure 4: An even-determined system with a unique solution.

Over-determined System

When the sample size is larger than the variable dimension i.e., $m > d$, we have an over-determined system. This means that we have more equations than unknowns. In this case, there is no exact solution for this set of equations. However, an approximated solution is yet available. If there exists a left-inverse of $\mathbf{X}$ such that $\mathbf{X}^{\dagger}\mathbf{X} = \mathbf{I}$, then by pre-multiplying both sides of (3.4) by the left-inverse $\mathbf{X}^{\dagger}$, we have in general, an approximated solution given by

$$\begin{aligned} \mathbf{X}^{\dagger}\mathbf{X}\boldsymbol{w} &= \mathbf{X}^{\dagger}\mathbf{y} \\ \Rightarrow \quad \hat{\boldsymbol{w}} &= \mathbf{X}^{\dagger}\mathbf{y}, \end{aligned} \tag{3.9}$$

where the left-inverse of $\mathbf{X}$ can be computed as $\mathbf{X}^{\dagger} = \left(\mathbf{X}^{T}\mathbf{X}\right)^{-1}\mathbf{X}^{T}$ if $\mathbf{X}^{T}\mathbf{X}$ is invertible.

Note: An exception is that when $\text{rank}(\mathbf{X}) = \text{rank}([\mathbf{X}, \mathbf{y}])$, the solution is exact.

Over-determined System

Example 3.2

Consider the following system of three equations with two unknowns,

$$
\begin{aligned}
w_1 + 2w_2 &= 1 && (3.10)\\
w_1 - 2w_2 &= 2 && (3.11)\\
2w_1 &= 1, && (3.12)
\end{aligned}
$$

which can be packed into matrix-vector form $\mathbf{X}\boldsymbol{w} = \mathbf{y}$ where

$$
\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 1 & -2 \\ 2 & 0 \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}.
$$

According to (3.9), and since $\mathbf{X}^T\mathbf{X}$ is invertible, we have

$$
\begin{aligned}
\hat{\boldsymbol{w}} &= \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y} \\
&= \begin{bmatrix} 6 & 0 \\ 0 & 8 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 2 \\ 2 & -2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.8333 \\ -0.2500 \end{bmatrix}, \qquad (3.13)
\end{aligned}
$$

Over-determined System

Example 3.2 cont'd

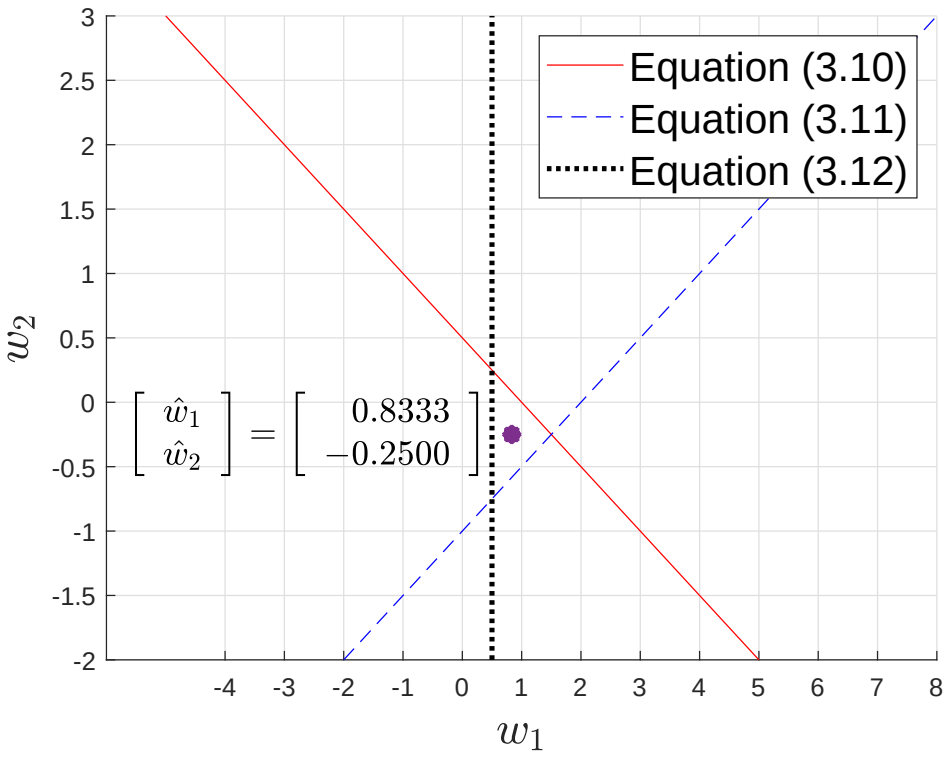

Figure 5: An over-determined system with an approximated solution.

Over-determined System

Example 3.3

Consider the following system of three equations with two unknowns,

$$
\begin{aligned}
w_1 + 2w_2 &= 1 && (3.14)\\
w_1 - 2w_2 &= 2 && (3.15)\\
2w_1 &= 3, && (3.16)
\end{aligned}
$$

which can be packed into matrix-vector form $\mathbf{X}\boldsymbol{w} = \mathbf{y}$ where

$$
\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 1 & -2 \\ 2 & 0 \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.
$$

Since $\mathbf{X}^T\mathbf{X}$ is invertible and $\text{rank}(\mathbf{X}) = \text{rank}([\mathbf{X}, \mathbf{y}])$, we have an exact solution as follows:

$$
\begin{aligned}
\hat{\boldsymbol{w}} &= \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y} \\
&= \begin{bmatrix} 6 & 0 \\ 0 & 8 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 2 \\ 2 & -2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1.50 \\ -0.25 \end{bmatrix}. && (3.17)
\end{aligned}
$$

Over-determined System

Example 3.3 cont'd

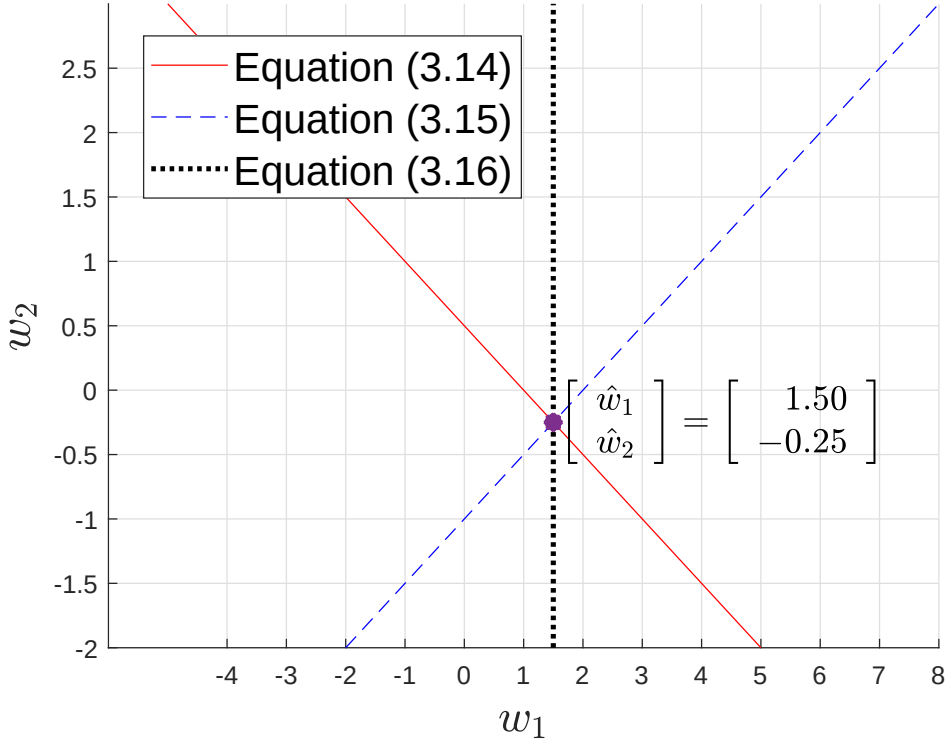

Figure 6: An over-determined system with an exact solution.

Under-determined System

When the sample size is smaller than the variable dimension i.e., $m < d$, we have an under-determined system. This means that we have fewer equations than unknowns. In this case, there are an infinite number of solutions for this set of equations. If there exists a right-inverse of $\mathbf{X}$ such that $\mathbf{X}\mathbf{X}^{\dagger} = \mathbf{I}$, then the d-vector

$$\hat{\boldsymbol{w}} = \mathbf{X}^{\dagger}\mathbf{y}, \tag{3.18}$$

which is one of the infinite solutions, satisfies the equation (3.4), i.e.,

$$\begin{aligned} \mathbf{X}\hat{\boldsymbol{w}} &= \mathbf{y} \\ \Rightarrow \quad \mathbf{X}\mathbf{X}^{\dagger}\mathbf{y} &= \mathbf{y} \\ \Rightarrow \quad \mathbf{y} &= \mathbf{y}. \end{aligned} \tag{3.19}$$

The right-inverse of $\mathbf{X}$ can be computed as $\mathbf{X}^{\dagger} = \mathbf{X}^{T}\left(\mathbf{X}\mathbf{X}^{T}\right)^{-1}$ when $\mathbf{X}\mathbf{X}^{T}$ is invertible.

Note: An exception for this under-determined case is that the system has no solution if it is inconsistent or $\mathbf{X}\mathbf{X}^{T}$ is not invertible (such as having all parallel planes in the system of equations).

Under-determined System

Example 3.4

Consider the following system of two equations with three unknowns,

$$w_1 + 2w_2 + 3w_3 = 1 \tag{3.20}$$

$$-w_1 - 2w_2 + 3w_3 = 2, \tag{3.21}$$

which can be packed into matrix-vector form $\mathbf{X}\boldsymbol{w} = \mathbf{y}$ where

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ -1 & -2 & 3 \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

According to (3.18), and since $\mathbf{X}\mathbf{X}^T$ is invertible, we have

$$\begin{aligned} \hat{\boldsymbol{w}} &= \mathbf{X}^T \left(\mathbf{X}\mathbf{X}^T\right)^{-1} \mathbf{y} \\ &= \begin{bmatrix} 1 & -1 \\ 2 & -2 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} 14 & 4 \\ 4 & 14 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -0.1 \\ -0.2 \\ 0.5 \end{bmatrix}. \end{aligned} \tag{3.22}$$

Under-determined System

Example 3.4 cont'd

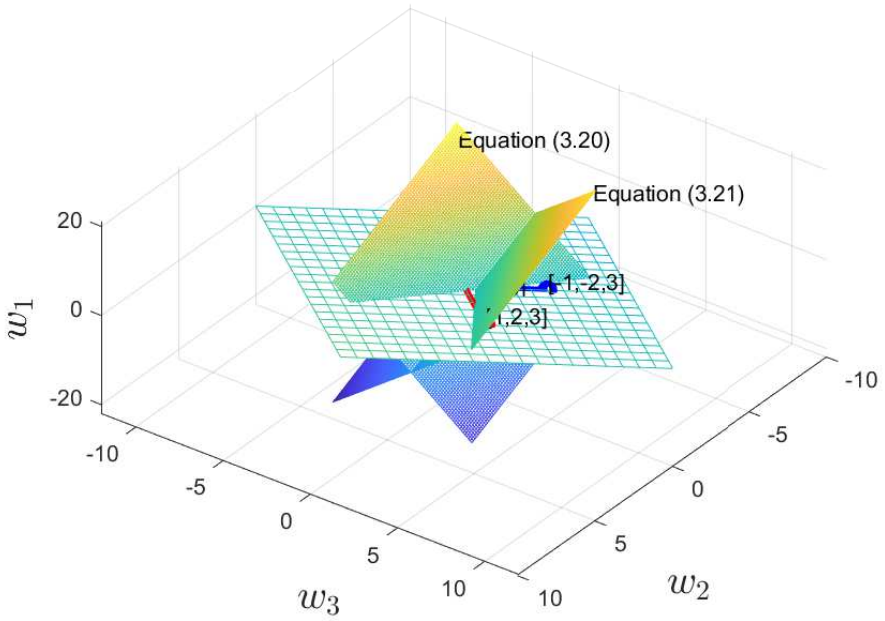

Figure 7: An under-determined system with a unique solution.

Under-determined System

Example 3.5

Consider the following system of two equations with three unknowns,

$$w_1 + 2w_2 + 3w_3 = 1 \tag{3.23}$$

$$-w_1 - 2w_2 - 3w_3 = 8, \tag{3.24}$$

which can be packed into matrix-vector form $\mathbf{X}\boldsymbol{w} = \mathbf{y}$ where

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ -1 & -2 & -3 \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 8 \end{bmatrix}.$$

This system is inconsistent because neither $\mathbf{X}^T\mathbf{X}$ nor $\mathbf{X}\mathbf{X}^T$ is invertible.

Under-determined System

Example 3.5 cont'd

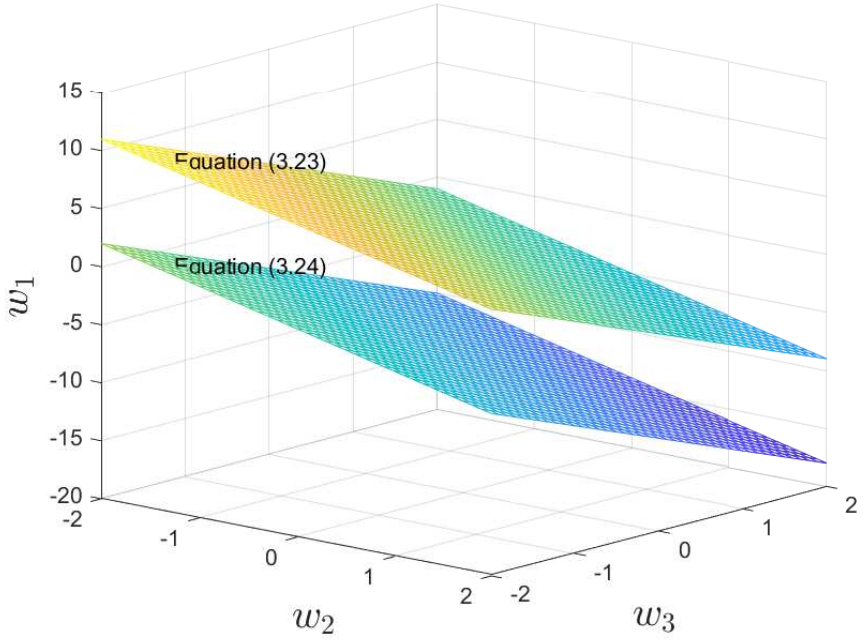

Figure 8: An inconsistent under-determined system.

Linear Model

Suppose there are d number of input features where they are packed as a vector given by $\boldsymbol{x} = [x_1, ..., x_d]^T$. Each feature vector can be indexed by a sample number $i \in \{1, ..., m\}$ for the pool of data, giving $\boldsymbol{x}_i = [x_{i,1}, ..., x_{i,d}]^T$. These feature samples can be stacked as a collection of vectors in matrix form as

$$\mathbf{X} = \begin{bmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_m^T \end{bmatrix} = [\mathbf{x}_1, \cdots, \mathbf{x}_d] = \overbrace{\begin{bmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{m,1} & \cdots & x_{m,d} \end{bmatrix}}^{\begin{bmatrix} \mathbf{x}_1 & \cdots & \mathbf{x}_d \end{bmatrix}} \Bigg\} \begin{bmatrix} \boldsymbol{x}_1^T \\ \vdots \\ \boldsymbol{x}_m^T \end{bmatrix}. \tag{3.25}$$

Correspondingly, the system response y can be indexed by the sample number giving y_i, $i \in \{1, ..., m\}$.

Linear Model

Based on these notations, a linear parametric model

$$g(\boldsymbol{w}, \boldsymbol{x}_i) = \sum_{j=1}^{d} w_j x_{i,j} = \boldsymbol{w}^T \boldsymbol{x}_i, \quad i = 1, \ldots, m, \tag{3.26}$$

can be used to model the system response or output as

$$y_i = g(\boldsymbol{w}, \boldsymbol{x}_i) + \epsilon_i = \sum_{j=1}^{d} w_j x_{i,j} + \epsilon_i, \quad i = 1, \ldots, m, \tag{3.27}$$

where ϵ_i is the modeling error. In a more compact manner, (3.27) can be re-written as

$$\mathbf{y} = \mathbf{X}\boldsymbol{w} + \boldsymbol{\epsilon}, \tag{3.28}$$

where $\mathbf{y} = [y_1, \cdots, y_m]^T$, $\boldsymbol{\epsilon} = [\epsilon_1, \cdots, \epsilon_m]^T$, $\mathbf{X}$ is defined in (3.25) and $\boldsymbol{w} = [w_1, \cdots, w_d]^T$.

Linear Model

In practice, an augmented model is preferred since an additional degree of freedom is available to deal specifically with the overall data offset. If not otherwise mentioned, the notation $\mathbf{X}$ shall take the regressor ot the augmented model as follows:

$$\mathbf{X} = \begin{bmatrix} 1 & \boldsymbol{x}_1^T \\ \vdots & \vdots \\ 1 & \boldsymbol{x}_m^T \end{bmatrix} = [\mathbf{1}, \mathbf{x}_1, \cdots, \mathbf{x}_d] = \overbrace{\begin{bmatrix} 1 & x_{1,1} & \cdots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \cdots & x_{m,d} \end{bmatrix}}^{\begin{bmatrix} \mathbf{x}_0 & \mathbf{x}_1 & \cdots & \mathbf{x}_d \end{bmatrix}}. \tag{3.29}$$

For such an augmented model, the corresponding *model coefficients* or *weight parameters* are denoted as $\boldsymbol{w} = [w_0, w_1, ..., w_d]^T \in \mathbb{R}^{d+1}$.

Generalized Linear Models

Transforming the input

The above linear model can be generalized to include transformed inputs. In other words, the inputs can be embedded under a nonlinear mapping with yet a linear relationship between the parameters and the embedment. Equation (3.26) can thus be generalized as

$$g(\boldsymbol{w}, \boldsymbol{x}_i) = \sum_{j=0}^{D} w_j p_j(\boldsymbol{x}_i) = \boldsymbol{w}^T \boldsymbol{p}(\boldsymbol{x}_i), \quad i = 1, \ldots, m, \tag{3.30}$$

where $p_j(\boldsymbol{x}_i)$, $j = 0, 1, \ldots, D$ are the transformed terms which can be linear or nonlinear with respect to $\boldsymbol{x}_i$. In general, the number of transformed terms D is different from the original feature size d. The vector $\boldsymbol{p}(\boldsymbol{x}_i)^T = [p_0(\boldsymbol{x}_i), p_1(\boldsymbol{x}_i), \cdots, p_D(\boldsymbol{x}_i)]$ is a packing of these transformed terms for the ith.

Generalized Linear Models

By stacking multiple samples of an augmented model in matrix form given by

$$\mathbf{P} = \overbrace{\begin{bmatrix} 1 & p_1(\boldsymbol{x}_1) & \cdots & p_D(\boldsymbol{x}_1) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & p_1(\boldsymbol{x}_m) & \cdots & p_D(\boldsymbol{x}_m) \end{bmatrix}}^{\begin{bmatrix} \mathbf{p}_0(\boldsymbol{x}) & \mathbf{p}_1(\boldsymbol{x}) & \cdots & \mathbf{p}_D(\boldsymbol{x}) \end{bmatrix}} \left\} \begin{bmatrix} \boldsymbol{p}(\boldsymbol{x}_1)^T \\ \vdots \\ \boldsymbol{p}(\boldsymbol{x}_m)^T \end{bmatrix}, \right. \tag{3.31}$$

a generalized linear system (with respect to (3.28)) can be written as

$$\mathbf{y} = \mathbf{P}\boldsymbol{w} + \boldsymbol{\epsilon}. \tag{3.32}$$

Here, we note that $\mathbf{P} \in \mathbb{R}^{m \times (D+1)}$ can be any arbitrary transformation matrix which maps the input feature space of $\mathbb{R}^d$ dimension to transformed feature space of $\mathbb{R}^{D+1}$ dimension. Popular choices for such transformation include polynomials, sigmoid function, Gaussian function and random projection. In the following sections, the various transformations are grouped into representative categories and discussed in greater detail.

Generalized Linear Models

Example 3.6

Consider the following samples of 2-dimensional input features (x_1, x_2):

features	sample 1	sample 2	sample 3	sample 4	sample 5	sample 6
x_1:	-1	-3	1	2	5	8
x_2:	1	-1	6	8	2	3

which can be packed in matrix form with inclusion of an intercept/bias term as

$$\mathbf{X} = \begin{bmatrix} 1 & -1 & 1 \\ 1 & -3 & -1 \\ 1 & 1 & 6 \\ 1 & 2 & 8 \\ 1 & 5 & 2 \\ 1 & 8 & 3 \end{bmatrix}. \tag{3.33}$$

Generalized Linear Models

Example 3.6 Cont'd

Suppose we have a transformation $p_1(\cdot) \triangleq e^{(\cdot)}$. Then the transformed features can be written in matrix form as

$$\mathbf{P} = \begin{bmatrix} 1 & e^{(-1)} & e^{(1)} \\ 1 & e^{(-3)} & e^{(-1)} \\ 1 & e^{(1)} & e^{(6)} \\ 1 & e^{(2)} & e^{(8)} \\ 1 & e^{(5)} & e^{(2)} \\ 1 & e^{(8)} & e^{(3)} \end{bmatrix}. \tag{3.34}$$

Suppose we have a transformation $p_1(\cdot) \triangleq (\cdot)^2$ and $p_2(\cdot) \triangleq \sin(\cdot)$. Then the transformed features can be written in matrix form as

$$\mathbf{P} = \begin{bmatrix} 1 & (-1)^2 & (1)^2 & \sin(-1) & \sin(1) \\ 1 & (-3)^2 & (-1)^2 & \sin(-3) & \sin(-1) \\ 1 & (1)^2 & (6)^2 & \sin(1) & \sin(6) \\ 1 & (2)^2 & (8)^2 & \sin(2) & \sin(8) \\ 1 & (5)^2 & (2)^2 & \sin(5) & \sin(2) \\ 1 & (8)^2 & (3)^2 & \sin(8) & \sin(3) \end{bmatrix}. \tag{3.35}$$

Generalized Linear Models

Transforming the output

For problems with binary outputs $y \in \{0, 1\}$, the logistic regression models the probabilities $\Pr(Y = 1|X = \boldsymbol{x}) = p$ and $\Pr(Y = 0|X = \boldsymbol{x}) = 1 - p$ (X and Y are random variables here) as a linear function of measurement $\boldsymbol{x} \in \mathbb{R}^d$. Here, the ratio between these output probabilities is a transformation that the probability value p is bounded within [0, 1]. The *logistic* (or *logit*) *transformation*, $\log(p/(1-p))$ (also known as the *logit link* function) is such a transformation that gives the desired property. A linear modeling based on the logistic output transformation can thus be written as

$$\log \frac{p}{1-p} = w_0 + \boldsymbol{w}^T \boldsymbol{x}. \tag{3.36}$$

Based on this output transformation, the probability of the class-1 output ($\Pr(Y = 1|X = \boldsymbol{x}) = p$) and the class-0 output ($\Pr(Y = 0|X = \boldsymbol{x}) = 1 - p$) can be respectively expressed in terms of the input feature vector $\boldsymbol{x}$ as

$$p = \frac{1}{1 + e^{-(w_0 + \boldsymbol{w}^T \boldsymbol{x})}}, \tag{3.37}$$

and

$$1 - p = \frac{e^{-(w_0 + \boldsymbol{w}^T \boldsymbol{x})}}{1 + e^{-(w_0 + \boldsymbol{w}^T \boldsymbol{x})}}. \tag{3.38}$$

Generalized Linear Models

For problems with multiple (N_c) number of categories, the logit model has the form

$$\begin{aligned}
\log \frac{\Pr(Y = 1 | X = \boldsymbol{x})}{\Pr(Y = N_c | X = \boldsymbol{x})} &= w_1^0 + \boldsymbol{w}_1^T \boldsymbol{x}, \\
\log \frac{\Pr(Y = 2 | X = \boldsymbol{x})}{\Pr(Y = N_c | X = \boldsymbol{x})} &= w_2^0 + \boldsymbol{w}_2^T \boldsymbol{x}, \\
&\vdots \\
\log \frac{\Pr(Y = N_c - 1 | X = \boldsymbol{x})}{\Pr(Y = N_c | X = \boldsymbol{x})} &= w_{(N_c-1)}^0 + \boldsymbol{w}_{(N_c-1)}^T \boldsymbol{x},
\end{aligned} \tag{3.39}$$

where the probability for each category is

$$\begin{aligned}
\Pr(Y = k | X = \boldsymbol{x}) &= \frac{e^{w_k^0 + \boldsymbol{w}_k^T \boldsymbol{x}}}{1 + \sum_{j=1}^{N_c - 1} e^{w_j^0 + \boldsymbol{w}_j^T \boldsymbol{x}}}, \; k = 1, \ldots, N_c - 1, \\
\Pr(Y = N_c | X = \boldsymbol{x}) &= \frac{1}{1 + \sum_{j=1}^{N_c - 1} e^{w_j^0 + \boldsymbol{w}_j^T \boldsymbol{x}}}.
\end{aligned} \tag{3.40}$$

In the above notations, w_j^0 denotes the bias term.

Full Multivariate Polynomials

According to the Weierstrass approximation theory, the polynomials can approximate any continuous function on a closed and bounded interval to any degree of accuracy. This means that multivariate polynomial regression provides an effective way to describe complex nonlinear input-output relationships.

A two-dimensional polynomial model (with x_1 and x_2 as the input feature dimensions) of second order (the highest power of x_1 and x_2 terms is 2) without any bias term can be written as

$$g(x_1, x_2) = x_1 + x_2 + x_1 x_2 + x_1^2 + x_2^2. \tag{3.41}$$

Full Multivariate Polynomials

Fig. 9 shows a mesh plot of this polynomial (3.41) which portrays a typical quadratic curvature with positive definite second derivative.

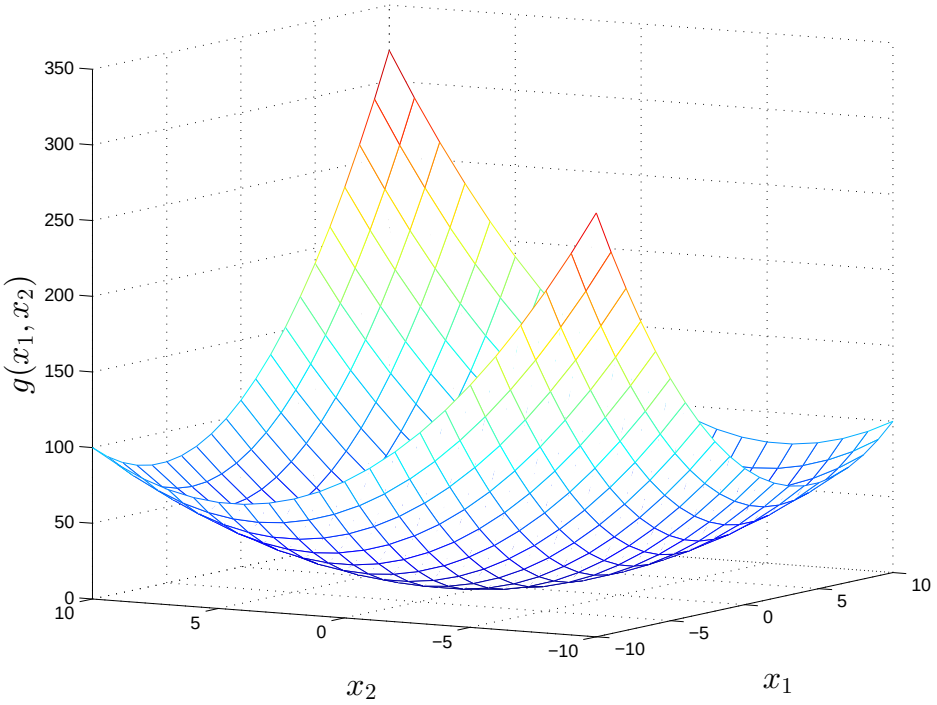

Figure 9: A second order polynomial model given by $g(x_1, x_2) = x_1 + x_2 + x_1x_2 + x_1^2 + x_2^2$.

Full Multivariate Polynomials

Fig. 10 show two plots of a third order polynomial model with different coefficient signs. These figures demonstrate a higher mapping complexity of the third order polynomials than that of the second order polynomials.

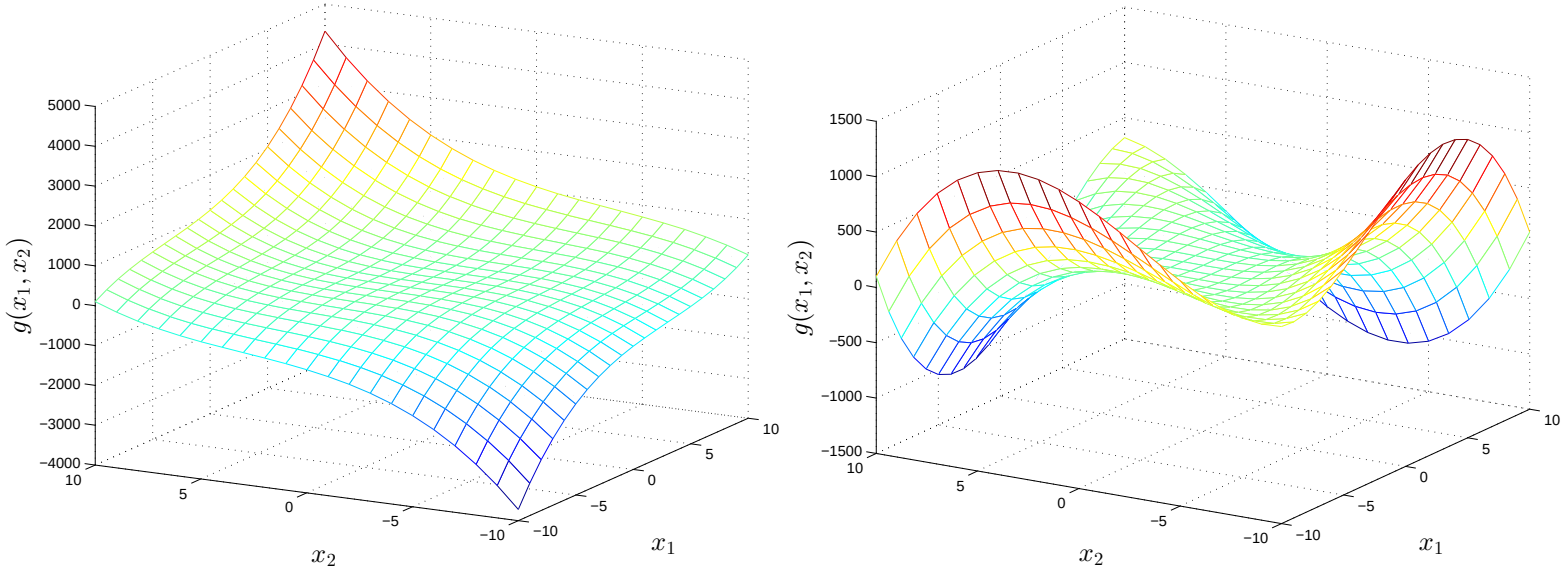

Figure 10: Left panel: A third order polynomial model given by $g(x_1, x_2) = x_1 + x_2 + x_1x_2 + x_1^2 + x_2^2 + x_1x_2^2 + x_2x_1^2 + x_1^3 + x_2^3$; Right panel: a third order polynomial model given by $g(x_1, x_2) = x_1 + x_2 + x_1x_2 + x_1^2 + x_2^2 + x_1x_2^2 + x_2x_1^2 - x_1^3 - x_2^3$.

Full Multivariate Polynomials

The general multivariate polynomial model can be expressed as

$$g(\boldsymbol{w}, \boldsymbol{x}) = \sum_{j=0}^{D} w_j x_1^{r_1} x_2^{r_2} \cdots x_d^{r_d} = \sum_{j=0}^{D} w_j p_j(\boldsymbol{x}), \tag{3.42}$$

where each of the summation term is call a *monomial* and the summation is taken over all non-negative integers $r_1, \ldots, r_d$ for which $\sum_{i=1}^{d} r_i \leqslant r$ with r being the order of the model. $\boldsymbol{w} = [w_0, w_1, \ldots, w_D]^T$ is the parameter vector to be estimated and each $p_j(\cdot)$ represents a monomial term which is a product among the elements of the input vector $\boldsymbol{x} = [1, x_1, \ldots, x_d]^T$.

Full Multivariate Polynomials

Example 3.7

Consider those samples of 2-dimensional input features (x_1, x_2) from the previous example:

features	sample 1	sample 2	sample 3	sample 4	sample 5	sample 6
x_1:	-1	-3	1	2	5	8
x_2:	1	-1	6	8	2	3

These data features can be expanded by a 2nd-order polynomial model $\boldsymbol{p} = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$ and packed in matrix form as

$$\mathbf{P} = \begin{bmatrix} 1 & -1 & 1 & (-1)^2 & (-1)\times(1) & (1)^2 \\ 1 & -3 & -1 & (-3)^2 & (-3)\times(-1) & (-1)^2 \\ 1 & 1 & 6 & (1)^2 & (1)\times(6) & (6)^2 \\ 1 & 2 & 8 & (2)^2 & (2)\times(8) & (8)^2 \\ 1 & 5 & 2 & (5)^2 & (5)\times(2) & (2)^2 \\ 1 & 8 & 3 & (8)^2 & (8)\times(3) & (3)^2 \end{bmatrix}. \tag{3.43}$$

Full Multivariate Polynomials

In Python, the full polynomials can be readily generated from the sklearn library function:

Example 3.7 cont'd

```
import numpy as np
X = np.array([[-1, 1], [-3, -1], [1, 6], [2, 8], [5, 2], [8, 3]])
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(2)
P = poly.fit_transform(X)
print(P)
[[ 1. -1.  1.  1. -1.  1.]
 [ 1. -3. -1.  9.  3.  1.]
 [ 1.  1.  6.  1.  6. 36.]
 [ 1.  2.  8.  4. 16. 64.]
 [ 1.  5.  2. 25. 10.  4.]
 [ 1.  8.  3. 64. 24.  9.]]
```

Full Multivariate Polynomials

- Each polynomial expansion term can be viewed as a new *axis*, *basis* or *representation* of the data points.
- The left panel of Fig. 11 shows 4 data points of two colors (red and blue). When these data are viewed in their original axes (x and y), the two red circles are not linearly separable from the two blue circles (i.e., unable to separate the data using a straight line or a plane). This problem is also known as the Exclusive-OR (XOR) problem where the response (output) is 'one' (this response has been indicated by red coloring of the circles in the left panel of Fig. 11) when the two binary inputs are different (i.e., $x \neq y$), and 'zero' (this response has been indicated by blue coloring of circles in the left panel of Fig. 11) when the two binary inputs are the same (i.e., $x = y$). However, when these data are projected onto the polynomial axis $z = xy$, we can see from the right panel of Fig. 11 that the projected blue circles (small blue circles) become linearly separable from the projected red circles (small red circles).

Full Multivariate Polynomials

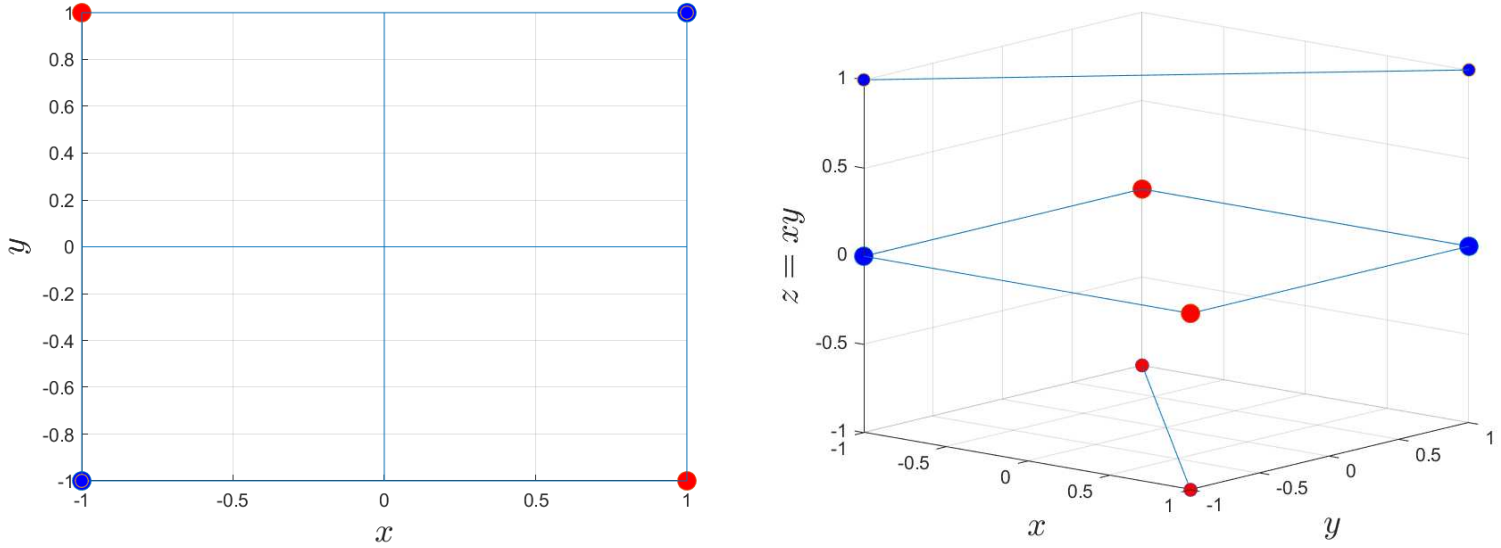

Figure 11: Left panel: the Exclusive OR (XOR) problem; Right panel: projected Exclusive OR (XOR) points.

Full Multivariate Polynomials

However, according to the chosen order r, the total number of monomial terms in $g(\boldsymbol{w}, \boldsymbol{x})$ is given by $D + 1$ where

$$D = \sum_{k=0}^{r} \frac{(k + d - 1)!}{k!(d - 1)!}, \tag{3.44}$$

with $0! = 1$.
This size of D can be approximately taken as an exponential growth of d^r with respect to the input dimension d and the model order r. For instance, a 10th-order model with 8 input features gives rise to a summation of 43758 monomial terms including the intercept. In view of this exponential growth of the number of parameters as well as the current limitation of memory and computing facility, the full multivariate polynomial model has yet to receive significant attention in applications.

Reduced Multivariate Polynomials

Motivation

- For regression applications, the above full multivariate polynomial model provides a versatile means for functional approximation.
- However, for classification applications, a model with full mapping capability may not be always necessary. This is because the main goal of a classifier is to find relevant decision boundary to segregate among data with different labels.
- And often enough, the complexity of such decision boundary cannot be too high where the generalization property may lost. In other words, an overly complex model may result in over-fitting of data and gives poor prediction for unseen data.

Reduced Multivariate Polynomials

To significantly reduce the huge number of terms in the multivariate polynomials, a Reduced multivariate polynomial Model (RM) has been proposed:

$$\begin{aligned} g_{RM}(\boldsymbol{w}, \boldsymbol{x}) &= \boldsymbol{w}^T p_{RM}(\boldsymbol{x}) \\ &= w_0 + \sum_{k=1}^{r} w_k \left(\sum_{j=1}^{l} x_j \right)^k + \sum_{k=1}^{r} \sum_{j=1}^{l} \beta_{k,j} x_j^k + \sum_{k=2}^{r} \left(\sum_{i=1}^{l} \gamma_{k,i} x_i \right) \left(\sum_{j=1}^{l} x_j \right)^{k-1} \\ & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad l, r \geqslant 2, \end{aligned} \tag{3.45}$$

where x_j, $j = 1, \ldots, l$ are the polynomial inputs, $\boldsymbol{w} = [w_0, w_k, \beta_{k,j}, \gamma_{k,i}]^T$, $j = 1, \ldots, l$, $k = 1, \ldots, r$ are the weighting coefficients to be estimated, and l, r correspond to input-dimension, order of system respectively. The number of terms in this model can be expressed as $K = 1 + r + l(2r - 1)$.

Reduced Multivariate Polynomials

In view of some redundant terms in RM, a refined model is constructed as:

$$g_{RM2}(\boldsymbol{w}, \boldsymbol{x}) = w_0 + \sum_{k=1}^{r} \sum_{j=1}^{l} \beta_{k,j} x_j^k + \sum_{k=2}^{r} \left(\sum_{i=1}^{l} \gamma_{k,i} x_i \right) \left(\sum_{j=1}^{l} x_j \right)^{k-1}, \quad (3.46)$$

where $l, r \geqslant 2$ and the total number of expansion terms is given by $K = 1 + l(2r - 1)$, which has r terms less than that in the original RM.

Reduced Multivariate Polynomials

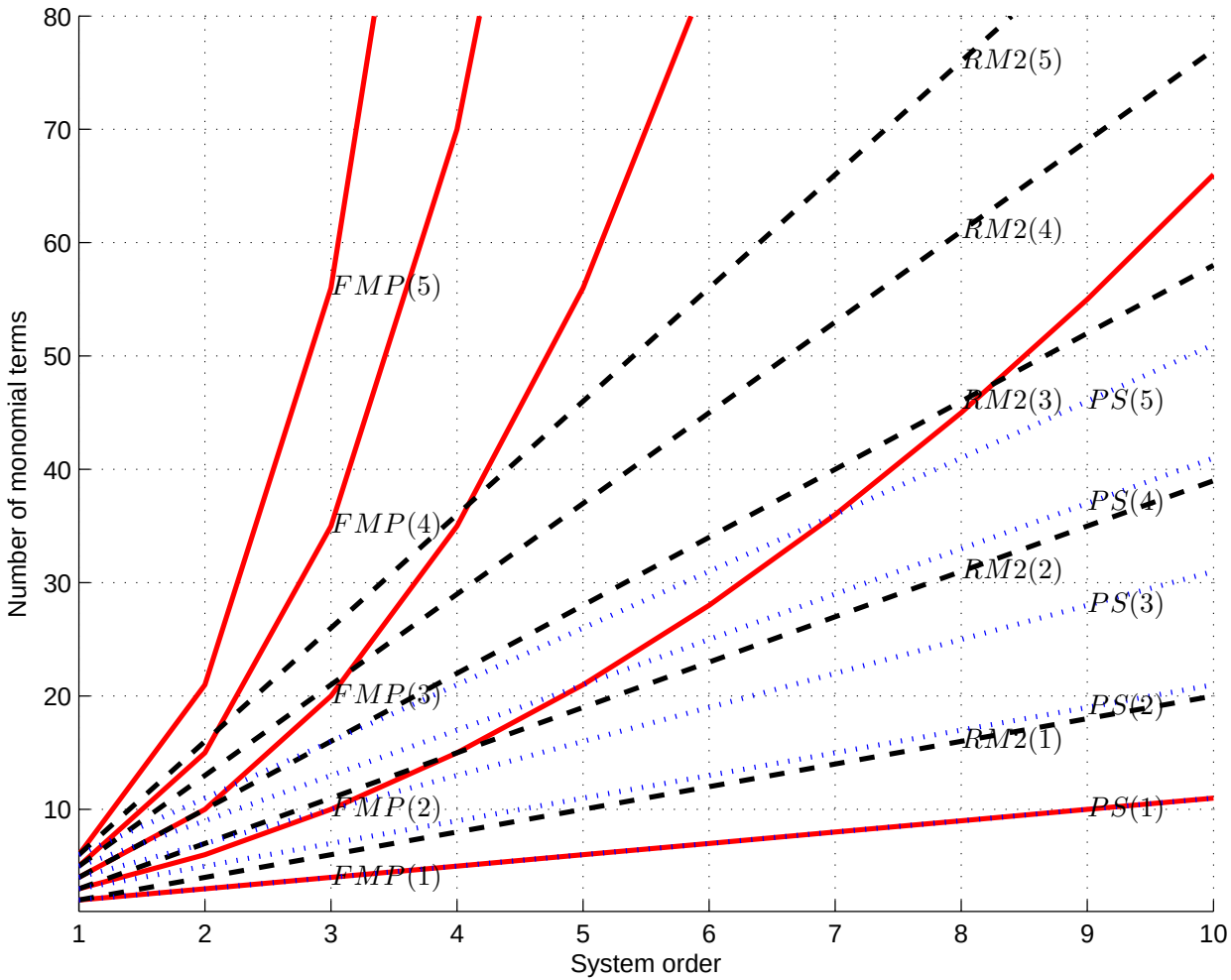

Figure 12: Number of monomial terms versus input dimension. The number within the parenthesis succeeding each model indicates the input size.

Reduced Multivariate Polynomials

Example 3.8

Consider those samples of 2-dimensional input features (x_1, x_2) from the previous example:

features	sample 1	sample 2	sample 3	sample 4	sample 5	sample 6
x_1:	-1	-3	1	2	5	8
x_2:	1	-1	6	8	2	3

The matrix expanded by the 2nd-order RM model can be generated using

```
X = [-1, 1; -3, -1; 1, 6; 2, 8; 5,2; 8,3]
P = RM(X,2)
P =
    1    -1     1     1     1     0     0
    1    -3    -1     9     1    12     4
    1     1     6     1    36     7    42
    1     2     8     4    64    20    80
    1     5     2    25     4    35    14
    1     8     3    64     9    88    33
```

Reduced Multivariate Polynomials

Example 3.8 cont'd

```
function P = RM(X,order)
% Build regressor matrix P (mxK):
%    order = desired order of approximation,
%    X = input matrix (mxl), K = number of parameters to be est.
%    m = number of data samples, l = input dimension.
[m,l] = size(X); MM1=[]; MM3=[]; Msum=sum(X,2);
for i=1:order
    for k=1:l
        M1(:,k)=X(:,k).^i;
        if (i>1)
            M3(:,k)=X(:,k).*Msum.^(i-1);
        end
    end
    MM1=[MM1,M1];
    if (i>1)
        MM3=[MM3,M3];
    end
end
P = [ones(m,1),MM1,MM3];
return;
```

Kernel Models

In machine learning, kernel methods refer to a class of algorithms for pattern analysis.

A representative example is the well known *support vector machine* (SVM) which hinges on the supporting vectors at data distribution boundary for classification decision.

The key idea of kernel method is to represent the raw data based on a *similarity function* called *kernel function* over *pairs of data points* whereas an explicit feature map, such as those projection and nonlinear transformation models $\boldsymbol{p}(\boldsymbol{x})$ mentioned in previous slides, needs not be known.

In other words, the estimation is made from data instances utilizing correlation (inner product) among data points with implicit parameters instead of estimating a fixed set of explicit data feature coordinates themselves.

Linear Kernel

Mathematically, a kernel function $k(\boldsymbol{x}_1, \boldsymbol{x}_2)$ which satisfies the Mercer's theorem[1], maps two data points $\boldsymbol{x}_1 \in \mathcal{X}$ and $\boldsymbol{x}_2 \in \mathcal{X}$ from their joint input space $\mathcal{X} \times \mathcal{X}$ onto another space $\mathcal{K}$, i.e., $k : \mathcal{X} \times \mathcal{X} \mapsto \mathcal{K}$.

In our applications, it is frequent to find $\mathcal{X} \subseteq \mathbb{R}^d$ and $\mathcal{K} \subseteq \mathbb{R}$, then $k(\boldsymbol{x}_1, \boldsymbol{x}_2)$ can be expressed as an inner product between the two points:

$$k(\boldsymbol{x}_1, \boldsymbol{x}_2) = \langle \boldsymbol{x}_1, \boldsymbol{x}_2 \rangle. \tag{3.47}$$

[1] According to AI Overview, Mercer's theorem is a functional analysis theorem that determines which functions can be used as kernel functions. Statement: A symmetric, continuous, and positive-definite kernel function K can be represented as a sum of a convergent sequence of product functions. Explanation: For a function K that takes two input vectors as arguments, K is a valid kernel if the symmetric matrix G is positive semi-definite. The matrix G is created by applying K to every possible pair of points in a dataset, where the i, jth element of G corresponds to $K(x_i, x_j)$.

Linear Kernel

According to Wahba's representer theorem, the minimizer of any arbitrary cost function plus a strict monotonic increasing transformation of the minimizer under a normed RKHS admits a representation of the form as a linear combination of kernelized training data points.

In simple terms, the model output g of any data point $\boldsymbol{x}$ can be expressed in terms of the training data $\boldsymbol{x}_i$, $i = 1, \ldots, m$ in inner product form given by

$$\begin{aligned} g(\boldsymbol{w}, \boldsymbol{x}) &= \sum_{i=1}^{m} \alpha_i \langle \boldsymbol{x}_i, \boldsymbol{x} \rangle \\ &= \sum_{i=1}^{m} \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}). \end{aligned} \tag{3.48}$$

Linear Kernel

By defining a mapping given by

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i \boldsymbol{x}_i = \sum_{i=1}^{m} \alpha_i \begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}_i, \tag{3.49}$$

where $\boldsymbol{x}_i$, $i = 1, \ldots, m$ are the training samples, we can relate the model in the *primal* parametric space ($\boldsymbol{w} \in \mathbb{R}^d$ or $\boldsymbol{w} \in \mathbb{R}^{d+1}$ when the bias term is included) to the model expressed in the *dual* parametric space ($\boldsymbol{\alpha} \in \mathbb{R}^m$):

$$\begin{aligned} g(\boldsymbol{w}, \boldsymbol{x}) &= \boldsymbol{w}^T \boldsymbol{x} \\ &= \left(\sum_{i=1}^{m} \alpha_i \boldsymbol{x}_i \right)^T \boldsymbol{x} \\ &= \sum_{i=1}^{m} \alpha_i \boldsymbol{x}_i^T \boldsymbol{x} = \sum_{i=1}^{m} \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}) \\ &= \boldsymbol{\alpha}^T \boldsymbol{k}(\boldsymbol{x}) = g(\boldsymbol{\alpha}, \boldsymbol{x}), \end{aligned} \tag{3.50}$$

where

$$\boldsymbol{\alpha} = [\alpha_1, \ldots, \alpha_m]^T, \tag{3.51}$$

and

$$\boldsymbol{k}(\boldsymbol{x}) = [k(\boldsymbol{x}_1, \boldsymbol{x}), \ldots, k(\boldsymbol{x}_m, \boldsymbol{x})]^T. \tag{3.52}$$

Linear Kernel

Here, we see a change of parameters based on the input feature space $\boldsymbol{w} \in \mathbb{R}^d$ or $\mathbb{R}^{d+1}$ to that based on the data sample space $\boldsymbol{\alpha} \in \mathbb{R}^m$ in (3.50) since $\boldsymbol{\alpha} = [\alpha_1, ..., \alpha_m]^T$ where m is the training sample size.

In other words, $\boldsymbol{\alpha}$ is the new vector of parameters to be estimated in (3.50) under the kernel paradigm.

Linear Kernel

Collectively, for m number of training points $\mathbf{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_m]^T \in \mathbb{R}^{m \times d}$ and target $\mathbf{y} = [y_1, \ldots, y_m]^T \in \mathbb{R}^m$, we can express the set of training prediction by

$$\begin{aligned}
\mathbf{y} &= \mathbf{g}(\boldsymbol{w}, \boldsymbol{x}) + \boldsymbol{\epsilon} \\
&= \mathbf{X}\boldsymbol{w} + \boldsymbol{\epsilon} \\
&= \begin{bmatrix} \boldsymbol{w}^T \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{w}^T \boldsymbol{x}_m \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \vdots \\ \epsilon_m \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}_1) \\ \vdots \\ \sum_{i=1}^m \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}_m) \end{bmatrix} + \boldsymbol{\epsilon} \\
&= \mathbf{X}\mathbf{X}^T \boldsymbol{\alpha} + \boldsymbol{\epsilon} = \mathbf{K}\boldsymbol{\alpha} + \boldsymbol{\epsilon}, \qquad (3.53)
\end{aligned}$$

where

$$\mathbf{K} = \mathbf{X}\mathbf{X}^T \qquad (3.54)$$

$$\boldsymbol{w} = \mathbf{X}^T \boldsymbol{\alpha}, \qquad (3.55)$$

with $\boldsymbol{\alpha} \in \mathbb{R}^m$ and $\mathbf{K} \in \mathbb{R}^{m \times m}$.

Polynomial Kernel

The main advantage of adopting a kernel model is its capability of embedding nonlinearity and dimensionality under the data sample space without explicit knowledge of the feature mapping coordinates. For example, an rth-order model adopting the polynomial kernel can be written as

$$g(\boldsymbol{w}, \boldsymbol{x}) = \sum_{i=1}^{m} \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}) \tag{3.56}$$

with

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = \langle \boldsymbol{\phi}(\boldsymbol{x}_i), \boldsymbol{\phi}(\boldsymbol{x}) \rangle = (a + b\langle \boldsymbol{x}_i, \boldsymbol{x} \rangle)^r, \tag{3.57}$$

where $a, b \in \mathbb{R}$ and an explicit expression for $\boldsymbol{\phi}$ needs not be known in practice.

Polynomial Kernel

An example for the unknown $\boldsymbol{\phi}$ in a second order polynomial model can be identified as

$$\boldsymbol{\phi}(\boldsymbol{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2,\ x_1^2,\ x_2^2, \sqrt{2}x_1x_2]^T, \tag{3.58}$$

where

$$\begin{aligned} k(\boldsymbol{x}_i, \boldsymbol{x}) &= \langle \boldsymbol{\phi}(\boldsymbol{x}_i), \boldsymbol{\phi}(\boldsymbol{x}) \rangle \\ &= (1 + \langle \boldsymbol{x}_i, \boldsymbol{x} \rangle)^2 \\ &= 1 + 2x_{i1}x_1 + 2x_{i2}x_2 + 1(x_{i1}x_1)^2 + 1(x_{i2}x_2)^2 + 2x_{i1}x_1x_{i2}x_2. \end{aligned} \tag{3.59}$$

Note: The monomials in this polynomial kernel come with a fixed combination of coefficients.

Gaussian Kernel

A model adopting the Gaussian kernel can be written as

$$g(\boldsymbol{w}, \boldsymbol{x}) = \sum_{i=1}^{m} \alpha_i k(\boldsymbol{x}_i, \boldsymbol{x}), \tag{3.60}$$

with

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = e^{-\|\boldsymbol{x}-\boldsymbol{x}_i\|^2/2\sigma^2}, \tag{3.61}$$

or

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = e^{-\gamma\|\boldsymbol{x}-\boldsymbol{x}_i\|^2}, \tag{3.62}$$

where $\sigma, \gamma \in \mathbb{R}$ correspond to the width of Gaussian function. When the square of the norm term is left out, the kernel is called an exponential kernel:

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = e^{-\|\boldsymbol{x}-\boldsymbol{x}_i\|/2\sigma^2}. \tag{3.63}$$

Creating More Kernels

Apart from the basic forms in above, more kernels can be created by a linear combination or product among kernels. Some examples are given below for immediate reference:

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = k_1(\boldsymbol{x}_i, \boldsymbol{x}) + k_2(\boldsymbol{x}_i, \boldsymbol{x}), \tag{3.64}$$

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = ak(\boldsymbol{x}_i, \boldsymbol{x}), \quad a \in \mathbb{R}^+, \tag{3.65}$$

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = k_1(\boldsymbol{x}_i, \boldsymbol{x})k_2(\boldsymbol{x}_i, \boldsymbol{x}), \tag{3.66}$$

$$k(\boldsymbol{x}_i, \boldsymbol{x}) = f(\boldsymbol{x}_i)f(\boldsymbol{x}). \tag{3.67}$$

Thank you!